

Python Dictionaries: A Reference

A dictionary in Python is a mutable collection of items stored as **key-value pairs** enclosed in curly braces (`{ }`). Keys act as unique custom lookups (like real-world words), while values represent the associated data (like definitions). Dictionaries do not look up items by numerical index positions.

1. Basic Core Mechanics

Tool / Syntax	Instruction & Core Mechanics	Step-by-Step Code & Output
<code>{key: value}</code> (Creation)	Defines a dictionary using key: value mappings separated by commas. Keys must be unique and immutable data types (like strings or integers).	<pre>>>> user = {"name": "Alice", "id": 101} >>> type(user) <class 'dict'> >>> empty = {} # Creates empty dictionary</pre>
<code>len()</code>	Counts the total number of key-value pairs inside the dictionary.	<pre>>>> data = {"UK": "London", "FR": "Paris"} >>> len(data) 2</pre>
<code>in</code>	Checks whether a specific key exists in the dictionary. It does not check values.	<pre>>>> user = {"name": "Alice", "role": "admin"} >>> "role" in user True >>> "admin" in user # Value search returns False False</pre>

2. Reading & Accessing Data

Syntax / Method	Instruction & Core Mechanics	Step-by-Step Code & Output
<code>dict[key]</code> (Square Brackets)	Looks up data by providing the explicit key inside square brackets. Throws a KeyError if the key does not exist.	<pre>>>> stock = {"apples": 50, "pears": 20} >>> stock["apples"] 50 >>> stock["bananas"] Traceback (most recent call last): KeyError: 'bananas'</pre>
<code>.get(key, default)</code>	Safely pulls data for a key. If the key is missing, instead of crashing, it returns None or a custom default value.	<pre>>>> stock = {"apples": 50} >>> stock.get("apples") 50 >>> stock.get("bananas") # Missing key, returns None</pre>

Syntax / Method	Instruction & Core Mechanics	Step-by-Step Code & Output
		<pre>>>> stock.get("bananas", 0) # Custom fallback value 0</pre>
Applied Case: Frequency Counter with <code>.get()</code>	Using <code>.get(item, 0) + 1</code> allows you to count items in a loop safely. If an item is new, it returns <code>0</code> and increments to <code>1</code> . If it exists, it increments its current count.	<pre>>>> items = ["apple", "banana", "apple"] >>> counts = {} >>> for x in items: ... counts[x] = counts.get(x, 0) + 1 ... >>> counts {'apple': 2, 'banana': 1}</pre>

3. Modifying & Adding Items

Syntax / Method	Instruction & Core Mechanics	Step-by-Step Code & Output
<code>dict[key] = val</code>	Assigns a value to a key. If the key already exists, its old value is overwritten. If it is missing, a new pair is created.	<pre>>>> profile = {"user": "Max"} >>> profile["user"] = "Sam" # Overwrites existing key >>> profile["status"] = "Active" # Adds brand new key >>> profile {'user': 'Sam', 'status': 'Active'}</pre>
<code>.pop(key)</code>	Removes a key and isolates its associated value into a variable. Throws a KeyError if the key isn't found.	<pre>>>> items = {"A": 1, "B": 2} >>> target = items.pop("A") >>> items {'B': 2} >>> target 1</pre>

4. Extracting Structures (For Loops)

Method	Instruction & Core Mechanics	Step-by-Step Code & Output
<code>.keys()</code>	Isolates and returns a view collection containing only the dictionary's keys.	<pre>>>> score = {"Max": 90, "Mia": 95} >>> list(score.keys()) ['Max', 'Mia']</pre>
<code>.values()</code>	Isolates and returns a view collection containing only the values.	<pre>>>> score = {"Max": 90, "Mia": 95} >>> list(score.values()) [90, 95]</pre>
<code>.items()</code>	Returns a sequence of key-value pairings structured as tuples . This is the	<pre>>>> score = {"Max": 90, "Mia": 95} >>> for name, points in score.items(): ... print(f"{name} got {points}")</pre>

Method	Instruction & Core Mechanics	Step-by-Step Code & Output
	standard pattern for running for loops over dictionaries.	<pre>... Max got 90 Mia got 95</pre>